

11 – OBJECT ORIENTED PROGRAMMING

11.a – Learning Outcomes

By the end of this chapter, you will be able to:

- **Define** what a class is and explain its role as a blueprint for creating objects.
- **Create** a Python class using the `class` keyword and follow PascalCase naming conventions.
- **Instantiate** objects from a class and explain how each object maintains its own state.
- **Implement** the `__init__()` method to initialize instance variables upon object creation.
- **Differentiate** between instance variables, class variables, instance methods, class methods, and static methods.
- **Apply** the `@classmethod` and `@staticmethod` decorators appropriately and explain when to use each.
- **Explain** the four pillars of Object-Oriented Programming: encapsulation, inheritance, polymorphism, and abstraction.
- **Implement** encapsulation using private variables and getter/setter methods.
- **Construct** class hierarchies using inheritance and override parent methods in child classes.
- **Demonstrate** polymorphism by defining methods with the same name across different classes.
- **Use** abstract base classes from the `abc` module to enforce consistent design.
- **Evaluate** when to use OOP principles to make programs more modular, reusable, and maintainable.

11.b – Introduction

Object-Oriented Programming (OOP) is a programming paradigm that organizes code around **objects** rather than functions and logic. An object bundles together **data** (attributes) and **behavior** (methods) into a single unit.

OOP is built around four fundamental principles — often called the **four pillars**:

1. **Encapsulation** – bundling data and methods, controlling access
2. **Inheritance** – creating new classes from existing ones
3. **Polymorphism** – same method name, different behavior
4. **Abstraction** – hiding complexity, exposing essentials

These principles help developers build programs that are **modular**, **reusable**, and **easier to maintain**.

Why OOP matters: As programs grow, managing complexity becomes the central challenge. OOP provides a structured way to model real-world entities, reduce duplication, and isolate changes so they don't ripple through the entire codebase.

11.c – Classes and Objects

A **class** can be considered as a software **blueprint** that defines the **properties** (attributes or states) and **behavior** (methods or actions) of an entity.

It serves as a template for creating an **object**, which is an individual **instance** of that class.

Each object created from a class maintains its **own state**, even though all share the same structure defined by the class.

11.c.1 – Defining a Python Class

You create a class using the `class` keyword. Note, class names in Python usually follows **PascalCase** naming style.

The following creates a minimalistic class

```
class Account:  
    pass
```

Note: The `pass` statement is a placeholder that allows the class body to be empty without causing a syntax error.

Interesting fact

Newer language uses the `new` operator to instantiate a class.

11.c.2 – Instantiating the Class

Create an object by calling the class like a function.

```
>>> obj = Account()  
>>> type(obj)  
<class '__main__.Account'>
```

11.c.3 – The `__init__` method

The `__init__()` method is called every time a new object is created from the class. It is referred to as the constructor in other languages.

Since the constructor is called immediately upon object creating, it is the best place to put initialization code or logic to make the object usable as soon as possible.

Interesting fact

The `__init__()` method is the constructor of the class, however it does not create an object, the runtime does that. The `__init__()` is invoked immediately after object creation.

```
class Account:
    def __init__(self, name, number, balance = 0):
        self.holder = name
        self.number = number
        self.balance = balance

>>> acct = Account('Arben Tapia', '1234', 100)
>>> acct.holder
'Arben Tapia'
>>> acct = Account('Ilia Nika', '1235', 200)
>>> acct.holder
'Ilia Nika'
```

11.c.4 – Instance Variables

Instance variable are values that are **unique to each object**. These are what makes each object different from other objects. Variable created with any instance method that are attached to the `'self'` pronoun are considered as instance variables.

Instance variables are most often declared in the `__init__()__()` because this method is called immediately upon object creation, hence any subsequent methods invoked will have access to those variables.

```
class Car:
    def __init__(self, range):
        self.range = range

    def drive(self):
```

```
        return f'I can drive {self.range}km'

>>> honda = Car(450)
>>> honda.drive()
'I can drive 450km'
```

11.c.5 – Instance Methods

Instance methods are functions declared inside the class that operate on instance data. They always take `self` as their first parameter, which refer to the current object.

```
class Car:
    def drive(self):
        return 'I can drive 500km'

>>> honda = Car()
>>> honda.drive()
I can drive 500km
```

11.c.6 – Class Variables

Class variables are shared by **all instances** of a class. Changing a class variable affects all objects.

```
class Person:
    species = 'Homo sapiens'      # Class variable

    def __init__(self, name):
        self.name = name         # Instance variable

>>> ilia = Person('Ilia')
>>> ana = Person('Ana')
>>> ilia.species
'Homo sapiens'
>>> ana.species
```

```
'Homo sapiens'

# Change affects all instances
>>> Person.species = 'Human'
>>> ilia.species
'Human'
```

Important: If you assign `ilia.species = 'Something'`, you create a **new instance variable** that shadows the class variable for that one object. To change the class variable for all objects, assign to `Person.species`.

11.c.7 – Class Methods

These are functions that belongs to a class rather than individual objects. Class methods operate on the class itself rather than instances. They're decorated with `@classmethod` and take `cls` as the first parameter.

Class methods are commonly used to modify class variables or **create alternative constructors**.

```
class Person:
    species = 'Homo sapiens'

    def __init__(self, name):
        self.name = name

    @classmethod
    def change_species(cls, new_species):
        cls.species = new_species

    @classmethod
    def create_anonymous(cls):
        return cls('Anonymous')    # Factory method

>>> Person.change_species('Human')
>>> Person.species
'Human'
```

```
>>> anonymous = Person.create_anonymous()
>>> anonymous.name
'Anonymous'
```

11.c.8 – Static Methods

Static methods are utility functions within a class that **don't access instance or class data**. They're decorated with `@staticmethod`.

```
class MathOperations:
    @staticmethod
    def add(x, y):
        return x + y

    @staticmethod
    def multiply(x, y):
        return x * y

>>> MathOperations.add(5, 3)
8
>>> MathOperations.multiply(4, 2)
8
```

11.c.9 – Key Concepts

Concept	Description	Example
Class	Blueprint for creating objects	<code>class Dog:</code>
Object	Instance of a class	<code>my_dog = Dog()</code>
Instance Variable	Unique to each object	<code>self.name = name</code>
Class Variable	Shared by all instances	<code>species = 'Canine'</code>

Instance Method	Operates on instance data	<code>def bark(self):</code>
Class Method	Operates on class data	<code>@classmethod</code>
Static Method	Utility function	<code>@staticmethod</code>
Inheritance	Child class inherits from parent	<code>class Puppy(Dog):</code>

11.c.10 – Comparison: Instance vs. Class vs. Static Methods

Feature	Instance Method	Class Method	Static Method
First parameter	<code>self</code>	<code>cls</code>	None
Decorator	None	<code>@classmethod</code>	<code>@staticmethod</code>
Accesses instance data?	Yes	No	No
Accesses class data?	Yes (via <code>self</code>)	Yes (via <code>cls</code>)	No
Call on	Instance	Class or instance	Class or instance
Common use	Behavior on object state	Factory methods, class state	Utility function

11.d – The Four Pillars of Object-Oriented Programming

Object-Oriented Programming (OOP) is built around four fundamental principles often referred to as the four pillars:

1. Encapsulation
2. Inheritance
3. Polymorphism
4. Abstraction

These principles help developers build programs that are **modular**, **reusable**, and **easier to maintain**.

11.b.1 – Encapsulation

Encapsulation is the practice of **bundling data (variables) and the methods that operate on that data** inside a class. It also involves **restricting direct access** to certain parts of an object.

In Python, encapsulation is commonly implemented using **private** or **protected** variables and **getter/setter** methods.

```
class BankAccount:

    def __init__(self, balance):
        self.__balance = balance    # private variable

    def deposit(self, amount):
        self.__balance += amount

    def get_balance(self):
        return self.__balance
```

Usage:

```
>>> acct = BankAccount(100)
>>> acct.deposit(50)
>>> acct.get_balance()
150
```

The variable `__balance` is intended to be accessed only through the class methods.

11.b.1.1 – Benefits of encapsulation:

1. Protects internal object data
2. Prevents accidental modification
3. Makes code easier to maintain

11.b.1.1 – Python naming conventions:

- `_variable` (single underscore) – "protected" by convention; a hint to other developers
- `__variable` (double underscore) – name-mangled to `__ClassName__variable`; not truly private but harder to access accidentally
- `__variable__` (double underscore on both sides) – reserved for Python's special methods (dunder methods)

11.b.2 – Inheritance

Inheritance allows a class to inherit attributes and methods from another class.

This promotes code reuse and allows developers to build hierarchies of related classes.

The class being inherited from is called the **parent class** (or superclass).

The class that inherits is called the **child class** (or subclass).

```
class Animal:
    def speak(self):
        print("Some sound")

class Dog(Animal):
    def speak(self):
        print("Bark")
```

Usage:

```
>>> d = Dog()
>>> d.speak()
Bark
```

Here, `Dog` inherits from `Animal` and **overrides** the `speak()` method.

11.b.2.1 – Benefits of inheritance:

1. Code reuse
2. Logical class hierarchies

3. Easier program organization

Note: Python supports **multiple inheritance** (a class can inherit from more than one parent), unlike some languages. Use it carefully to avoid the "diamond problem."

11.b.3 – Polymorphism

Polymorphism means “many forms.” It allows different classes to implement methods with the **same name but different behavior**.

```
class Dog:
    def speak(self):
        return "Bark"

class Cat:
    def speak(self):
        return "Meow"
```

Usage:

```
animals = [Dog(), Cat()]

for animal in animals:
    print(animal.speak())
```

Output:

```
Bark
Meow
```

Even though both classes define a method named `speak()`, each object behaves differently.

11.b.3.1 – Benefits of polymorphism:

1. Flexible code
2. Easier to extend programs
3. Supports interchangeable objects

Note: Polymorphism in Python is often achieved through **duck typing** — "If it walks like a duck and quacks like a duck, it's a duck." Python doesn't require a common parent class for polymorphism to work; it only requires that the object has the method being called.

11.b.4 – Abstraction

Abstraction means hiding complex implementation details and exposing only the essential features of an object.

Python often implements abstraction using **abstract base classes** from the `abc` module.

Example:

```
from abc import ABC, abstractmethod

class Shape(ABC):

    @abstractmethod
    def area(self):
        pass
```

Child classes must implement the abstract method:

```
class Rectangle(Shape):

    def __init__(self, width, height):
        self.width = width
        self.height = height

    def area(self):
        return self.width * self.height
```

Usage:

```
>>> r = Rectangle(5, 3)
>>> r.area()
15
```

The Shape class defines what must be implemented, but not how it should be implemented.

11.b.4.1 – Benefits of abstraction:

1. Simplifies modeling, designing, implementing and testing complex systems
2. Forces consistent design
3. Improves maintainability

Important: You cannot instantiate an abstract class directly. Attempting to do so raises a `TypeError`. Abstract classes exist only to be inherited from.

Summary of the Four Pillars

Pillar	Description	Key Idea
Encapsulation	Bundling data and methods together and controlling access	Protect internal state
Inheritance	Creating new classes from existing ones	Reuse and extend
Polymorphism	Same method name behaving differently across classes	One interface many form
Abstraction	Hiding complexity and exposing essential features	Define what, not how

11.e – Summary

This chapter introduced Object-Oriented Programming (OOP), a paradigm that organizes code around objects that bundle data and behavior.

Key takeaways:

- **Classes** are blueprints that define attributes (data) and methods (behavior). **Objects** are individual instances of a class.
- `__init__()` is the constructor — called automatically when an object is created — and is the best place to initialize instance variables.
- **Instance variables** are unique to each object; class variables are shared by all instances.
- **Instance methods** take `self` and operate on instance data. Class methods take `cls`, are decorated with `@classmethod`, and operate on class data. Static methods

take no special first parameter, are decorated with `@staticmethod`, and are utility functions.

- **Encapsulation** bundles data and methods and restricts direct access using private variables and getter/setter methods.
- **Inheritance** allows a child class to inherit attributes and methods from a parent class, promoting code reuse.
- **Polymorphism** allows different classes to implement methods with the same name but different behavior.
- **Abstraction** hides complex implementation details and exposes only essential features, often using abstract base classes from the `abc` module.
- **The four pillars** — encapsulation, inheritance, polymorphism, and abstraction — work together to make programs modular, reusable, and maintainable.

11.e.1 – Quick Reference: Defining a Class

```
class ClassName:
    class_variable = value          # Shared by all instances

    def __init__(self, param):      # Constructor
        self.instance_var = param  # Instance variable

    def instance_method(self):      # Operates on instance
        return self.instance_var

    @classmethod
    def class_method(cls):          # Operates on class
        return cls.class_variable

    @staticmethod
    def static_method():            # Utility function
        return "No access to instance or class"
```

11.f – Non coding exercises

1. What is a class?
2. What is an object?
3. What is the difference between a class and an object?
4. What naming convention is used for class names in Python?
5. What is the purpose of the `__init__()` method?
6. What is another name for the `__init__()` method in other languages?
7. What is an instance variable?
8. What is a class variable?
9. What is the difference between an instance variable and a class variable?
10. What is an instance method?
11. What is the first parameter of an instance method?
12. What is a class method?
13. What decorator is used for a class method?
14. What is the first parameter of a class method?
15. What is a static method?
16. What decorator is used for a static method?
17. What is the difference between a class method and a static method?
18. When would you use a class method instead of an instance method?
19. When would you use a static method?
20. What are the four pillars of OOP?
21. What is encapsulation?
22. How is encapsulation implemented in Python?
23. What is the difference between `_variable` and `__variable` in Python?
24. What is inheritance?
25. What is the parent class called? What is the child class called?
26. What does it mean to "override" a method?
27. What is polymorphism?
28. How does Python's duck typing relate to polymorphism?
29. What is abstraction?
30. What module provides abstract base classes in Python?
31. What decorator is used to define an abstract method?
32. Can you instantiate an abstract class directly? Why or why not?
33. True or false: A class can have both instance variables and class variables.
34. True or false: Changing a class variable affects all instances.
35. True or false: A static method can access instance variables.
36. True or false: A class method can access class variables.
37. True or false: Inheritance promotes code reuse.
38. True or false: Polymorphism requires a common parent class in Python.
39. True or false: Encapsulation protects internal object data.
40. True or false: Abstraction hides complexity and exposes essential features.

Appendix

Docstring